



UNIVERSITÀ
DI TORINO

COMBINING FUZZY INFORMATION FROM MULTIPLE SYSTEMS

Alessandro Zappatore

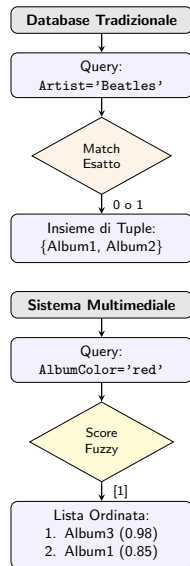
UNIVERSITÀ DEGLI STUDI DI TORINO

10/07/2026

Database Tradizionali vs. Sistemi Multimediali



- **Database Tradizionali** (es. Relazionali): Il risultato di una query (es. Artist='Beatles') è un set ben definito di valori. L'oggetto soddisfa la query oppure no.
- **Sistemi Multimediali** (es. QBIC): Il risultato di una query basata sul contenuto (es. AlbumColor='red') è una lista ordinata basata sul livello di "somiglianza".
- **La Sfida:** Come valutiamo una query congiunta come Artist='Beatles' $\wedge$ AlbumColor='red'? Serve una semantica in grado di risolvere questa discrepanza.



Garlic adotta il concetto di "**insieme graduato**" o **insieme fuzzy**.

- Un insieme graduato è composto da coppie (x, g) , dove x è l'oggetto valutato e g è il suo grado o "score".
- Il grado è un numero reale nell'intervallo.
- Un grado pari a 1 indica una corrispondenza perfetta (es. query relazionali tradizionali hanno gradi di 0 o 1), mentre valori intermedi rappresentano corrispondenza parziali.
- Insieme fuzzy:

$$S = \{(x, g) \mid x \in \text{Objects}, g \in [0, 1]\}$$

- Quando si combinano più query (es. una congiunzione di più attributi), Garlic si appoggia inizialmente alle regole standard della logica fuzzy (regole di Zadeh).
- Queste regole garantiscono la compatibilità con la logica proposizionale classica per gradi estremi (0 e 1) e preservano l'equivalenza logica (Teorema di Bellman-Giertz).
- **Congiunzione (AND):** $\mu_{A \wedge B}(x) = \min\{\mu_A(x), \mu_B(x)\}$
- **Disgiunzione (OR):** $\mu_{A \vee B}(x) = \max\{\mu_A(x), \mu_B(x)\}$
- **Negazione (NOT):** $\mu_{\neg A}(x) = 1 - \mu_A(x)$

- La congiunzione può essere valutata tramite una qualsiasi "Norma Triangolare" (T-norm), una classe di funzioni di aggregazione.
- Proprietà di una T-norm $t(x_1, x_2)$:
 - ① **Conservazione logica:** $t(0, 0) = 0$; $t(x, 1) = x$.
 - ② **Monotonicità:** All'aumentare dei gradi delle sotto-query, il grado totale non può diminuire.
 - ③ **Strettezza (Strictness):** Il voto complessivo è il massimo (1) solo se tutti gli argomenti hanno il voto massimo.
- **Importanza:** L'efficienza dell'algoritmo di Fagin dipenderà dalla monotonicità (per il limite superiore) e dalla strettezza (per il limite inferiore):
 - Monotonicità: $t(x_1, \dots, x_m) \leq t(x'_1, \dots, x'_m)$ se $x_i \leq x'_i$ per ogni i .
 - Strettezza: $t(x_1, \dots, x_m) = 1 \iff x_i = 1$ per ogni i .

- **Teorema 3.1:** Le uniche funzioni di aggregazione per valutare la congiunzione $\wedge$ e la disgiunzione $\vee$ che preservano l'equivalenza logica delle query e che sono monotone nei loro argomenti sono, rispettivamente, il `min` e il `max`.

L'interfaccia verso i Sottosistemi (Sorted vs Random Access)

Per ottimizzare il reperimento dei "top k " risultati senza leggere tutto il database, Garlic ha due modi per accedere ai dati nei vari sottosistemi (es. QBIC):

- **Accesso Ordinato:** Garlic chiede al sottosistema di restituire gli oggetti uno per uno, in ordine decrescente rispetto al loro grado di corrispondenza.
- **Accesso Casuale:** Garlic chiede al sottosistema di fornire grado specifico di un determinato oggetto noto.

- Un insieme X è chiuso verso l'alto (upwards closed) rispetto a una query Q se, per ogni $x \in X$, qualsiasi oggetto y con un grado maggiore di x appartiene anch'esso a X .
- **L'intuizione Chiave:** Se interroghiamo ogni sottosistema tramite accesso ordinato fino a ottenere k oggetti presenti in tutte le liste analizzate, siamo certi che la soluzione ottimale non può nascondersi negli oggetti ancora non esaminati.

L'Algoritmo $\mathcal{A}_0$ (L'Algoritmo di Fagin) - Fase 1

L'Algoritmo $\mathcal{A}_0$ valuta le congiunzioni monotone e restituisce i top k oggetti.

Fase 1: Accesso Ordinato (Sorted Access)

- ➊ **Lettura Parallela:** Si interrogano i sottosistemi contemporaneamente, prelevando oggetti uno ad uno in ordine decrescente di punteggio.
- ➋ **Condizione di Arresto:** L'algoritmo si ferma *solo* quando l'intersezione degli oggetti estratti da tutti i sottosistemi contiene almeno k "match" (oggetti estratti in tutti i sistemi).
- **Formalmente:** L'arresto avviene alla profondità T in cui:

$$\left| \bigcap_{i=1}^m X_{iT} \right| \geq k$$

(dove X_{iT} rappresenta l'insieme dei primi T oggetti del sottosistema i).

Fase 2: Accesso Casuale (Random Access)

- Per **tutti** gli oggetti estratti finora (ovvero l'unione $\bigcup_{i=1}^m X_{iT}$, non solo i k nell'intersezione), esegui un accesso casuale sui sottosistemi mancanti per ottenere tutti i gradi esatti $\mu_{A_j}(x)$.

Fase 3: Calcolo e Restituzione

- ➊ **Aggregazione:** Calcola il grado complessivo per ogni oggetto valutato usando la T-norma t :

$$\mu_Q(x) = t(\mu_{A_1}(x), \dots, \mu_{A_m}(x))$$

- ➋ **Ordinamento:** Ordina la lista e isola l'insieme Y contenente i migliori k oggetti.

L'Algoritmo $\mathcal{A}'_0$ per la Logica Fuzzy Standard (Min)

Quando la funzione di aggregazione è strettamente il $\min$, l'algoritmo $\mathcal{A}_0$ può essere ottimizzato per ridurre drasticamente i costosi accessi casuali.

- 1 **Trovare il minimo nell'intersezione:** Tra gli oggetti presenti nell'intersezione (i match presenti in tutte le liste estratte), si individua l'oggetto x_0 che ha il voto complessivo minimo g_0 .
- 2 **Individuare il Sistema Critico:** Si identifica il sottosistema i_0 che ha determinato questo punteggio minimo assegnando esattamente il grado g_0 a x_0 .
- 3 **Limitare i Candidati:** Invece di eseguire l'accesso casuale su *tutti* gli oggetti estratti, l'algoritmo valuta **solo** i "candidati", ovvero quegli oggetti estratti dal sistema critico i_0 che hanno mostrato un grado $\geq g_0$.

L'Algoritmo $\mathcal{B}_0$ per l'Operatore di Massimo (OR)

Se la query è una disgiunzione (Standard Fuzzy OR, aggregazione max), la complessità crolla e la strategia cambia radicalmente.

- ➊ **Solo Accesso Ordinato:** L'algoritmo si limita a prelevare esattamente i top k oggetti da *ciascun* degli m sottosistemi.
- ➋ **Zero Accessi Casuali:** Questo algoritmo non effettua **nessun** accesso casuale.
- ➌ **Fase di Calcolo:** Si valutano solo i (massimo) $m \cdot k$ oggetti estratti. Per ogni oggetto, il suo voto complessivo è semplicemente il punteggio massimo registrato tra le liste in cui è apparso nei primi k . Si restituiscono i migliori k globali.

- **Teorema 4.2:** Per ogni query monotona, l'algoritmo $\mathcal{A}_0$ restituisce correttamente le top k risposte.
 - *Dimostrazione:* Sia Y l'insieme delle k risposte trovate. Se esistesse un oggetto $z \notin Y$ con punteggio maggiore di un oggetto $y \in Y$, per monotonicità z dovrebbe avere un grado superiore a qualche oggetto x presente nell'intersezione trovata nella fase ordinata. Poiché le liste sono chiuse verso l'alto, z dovrebbe essere già stato estratto in tutte le liste, e quindi far parte dei candidati valutati, contraddicendo il fatto che $z \notin Y$.
- **Teorema 4.4:** Nel caso della congiunzione fuzzy standard ($\min$), l'algoritmo $\mathcal{A}'_0$ restituisce le top k risposte corrette.
 - *Dimostrazione:* Simile alla precedente, ma sfrutta la Proposizione 4.3: se z fosse un risultato migliore non ancora selezionato, il suo punteggio dovrebbe per forza superare la soglia minima trovata nella specifica lista i_0 che ha causato l'arresto, dimostrando che z deve necessariamente trovarsi tra i candidati limitati di $\mathcal{A}'_0$.

- **Teorema 4.5:** Nel caso della disgiunzione fuzzy standard ($\max$), l'algoritmo $\mathcal{B}_0$ è corretto.
 - *Dimostrazione:* Dato che l'operatore massimo dipende solo dal picco più alto tra le liste: prelevare i top k da ciascuna lista garantisce che nessun oggetto non letto possa avere un grado maggiore dei k migliori globali.

Come misuriamo l'efficienza degli algoritmi per Garlic? Fagin propone il **Middleware Cost**.

Il calcolo si basa sull'uso degli accessi al DB: non contano i cicli CPU, ma le richieste (I/O).

- **Ipotesi Probabilistica:** Per l'analisi, si presuppone che le sotto-query $\mathcal{A}_1, \dots, \mathcal{A}_m$ siano **indipendenti** e che gli ordinamenti degli oggetti siano generati casualmente e con eguale probabilità.
 - Costo = $c_1 \cdot S + c_2 \cdot R$
 - S = Costo di Sorted Access, R = Costo di random Access.

Per l'analisi delle performance, Fagin astrae il database originale attraverso due concetti matematici chiave:

- **Scoring Database ($\mathcal{D}$):** È una funzione che associa a ciascuna delle m liste (query atomiche) un *insieme graduato* di N oggetti (i valori esatti di appartenenza).
- **Skeleton ($\mathcal{S}$):** È un'astrazione dell'ordine spaziale. Associa a ciascuna delle m liste una permutazione degli oggetti da 1 a N .
- **Coerenza (Consistency):** Un database $\mathcal{D}$ è *coerente* con uno skeleton $\mathcal{S}$ se l'ordine imposto da $\mathcal{S}$ dispone gli elementi del database $\mathcal{D}$ esattamente in ordine decrescente rispetto al loro grado.

Formalizzazione dei Costi (Middleware e Unweighted)

Gli algoritmi vengono eseguiti considerando tutte le possibili configurazioni di punteggi. Vengono definiti tre tipi di costo, valutando il caso peggiore sui gradi, ma fissando l'ordine:

- $sortedcost(\mathcal{A}, \mathcal{S})$: Il costo di accesso ordinato dell'algoritmo $\mathcal{A}$ sullo skeleton $\mathcal{S}$. Corrisponde al caso peggiore (massimo) degli accessi sequenziali su tutti i possibili *scoring database* coerenti con $\mathcal{S}$.
- $sumcost(\mathcal{A}, \mathcal{S})$: Il costo middleware non pesato (*unweighted middleware cost*), ovvero la somma pura degli accessi ordinati e casuali ($S + R$).
- $cost(\mathcal{A}, \mathcal{S})$: Il *middleware cost* effettivo, ovvero la somma pesata $c_1 \cdot S + c_2 \cdot R$, strettamente delimitata matematicamente dal $sumcost$.

Preparazione all'Ottimalità: Il Lemma 5.1

Prima di dimostrare la performance sublineare, serve prevedere la dimensione dell'intersezione tra le liste caricate in memoria.

Lemma 5.1: Sia B_1 un insieme fissato di l_1 elementi e B_2 un insieme scelto *casualmente* di l_2 elementi tra N totali.

- Il valore atteso (media) della dimensione della loro intersezione $B = B_1 \cap B_2$ è $M = \frac{l_1 \cdot l_2}{N}$.
- Se $l_1 \leq \frac{N}{10}$, la probabilità che l'intersezione sia *minore o uguale alla metà del valore atteso* decade esponenzialmente:

$$\Pr \left[|B| \leq \frac{M}{2} \right] < e^{-\frac{M}{10}}$$

Dimostrazione del Lemma 5.1: I 4 Processi Probabilistici

La dimostrazione modella l'estrazione come dei "lanci di monete" e usa 4 processi stocastici per abbassare progressivamente le probabilità e applicare il limite di Chernoff:

- **Processo 1:** Modella l'intersezione reale. La probabilità che il prossimo elemento faccia parte di B_2 varia dinamicamente (es. senza rimpiazzo) a seconda dei successi passati.
- **Processo 2:** Applica un *limite inferiore artificiale* alla probabilità di successo del Processo 1, garantendo che le probabilità combacino per $h \leq a$ (numero critico di successi).
- **Processo 3:** Semplifica il calcolo rendendo la probabilità di successo **costante** $\frac{l_2 - a}{N - a}$ per tutti i lanci, sottostimando deliberatamente le performance del Processo 2.
- **Processo 4:** Abbassa ulteriormente la probabilità costante a $\frac{19}{20} \frac{l_2}{N}$. Questo rende banale l'applicazione del *Chernoff Bound*, che ci garantisce il limite esponenziale $e^{-\frac{M}{10}}$.

- **Teorema 5.3:** Assumendo che le m query atomiche siano indipendenti, il middleware cost per l'algoritmo $\mathcal{A}_0$ è $O(N^{(m-1)/m} \cdot k^{1/m})$ con probabilità arbitrariamente alta.
 - *Dimostrazione:* L'autore calcola la probabilità che, fermando la lettura a una profondità $T = c \cdot N^{(m-1)/m} \cdot k^{1/m}$, l'intersezione delle m liste contenga meno di k elementi. Usando l'induzione e il bound di Chernoff, dimostra che all'aumentare della costante c , la probabilità di fallire nel trovare i k match decade esponenzialmente. Pertanto, con probabilità vicina a 1, l'algoritmo si ferma entro il limite T ipotizzato.
- **Teorema 5.4:** Esiste un algoritmo per trovare le top k risposte a una query monotona con un costo $O(N^{(m-1)/m} \cdot k^{1/m})$.
 - *Dimostrazione:* Segue direttamente dalla combinazione del Teorema 4.2 e del Teorema 5.3.

Un Limite Insuperabile per Funzioni Strette

Si può fare meglio dell'Algoritmo $\mathcal{A}_0$? No.

- Il Teorema 6.4 e 6.5 dimostrano che per le funzioni **strette** (strict), nessun algoritmo corretto in grado di trovare i top k può avere un costo asintoticamente minore.
- $\mathcal{A}_0$ raggiunge quindi il limite teorico assoluto, ed è ottimale a meno di una costante.
- Eccezione: Funzioni non-strette (come il Max o la Mediana) non soffrono di questo limite e possono essere risolte molto più velocemente (costo non dipendente da N).

- **Teorema 6.4 e 6.6:** Per una funzione di aggregazione strict, nessun algoritmo corretto può garantire un costo (né totale né solo in accessi ordinati) asintoticamente inferiore a $O(N^{(m-1)/m} \cdot k^{1/m})$.
 - *Dimostrazione:* Se un algoritmo $\mathcal{A}$ analizzasse un numero di oggetti significativamente minore di T , l'intersezione attesa degli oggetti visti in tutte le liste sarebbe insufficiente (meno di k). Il lemma 6.2 dimostra che, in questa situazione, ci sarebbe almeno un oggetto "intoccato" (mai letto dal database). Un database "malevolo" potrebbe assegnare a questo oggetto non visto un grado pari a 1 in tutte le liste, rendendolo la risposta corretta, che l'algoritmo però mancherebbe, fallendo così il requisito di correttezza.
- **Teorema 6.5:** Il costo per risolvere query monotone e strette indipendenti è strettamente $\theta(N^{(m-1)/m} \cdot k^{1/m})$.
 - *Dimostrazione:* Ottenuta unendo matematicamente i limiti superiori (Teorema 5.3) e inferiori (Teorema 6.4), dimostrando che $\mathcal{A}_0$ è l'algoritmo ottimale per questa classe di problemi.

Le Query "Difficili": Correlazione Negativa

Cosa succede se le query sono negativamente correlate?

- Fagin introduce il caso limite della query $Q \wedge \neg Q$.
- **Logica Booleana:** Questo è un paradosso insoddisfacibile.
- **Logica Fuzzy:** Un elemento con grado 0.5 per Q avrà grado 0.5 per $\neg Q$, portando il voto complessivo a 0.5 (usando l'operatore di min).
- **Teorema 7.1:** Il costo middleware per trovare la migliore risposta ($k = 1$) alla congiunzione fuzzy standard $Q \wedge \neg Q$ è necessariamente lineare $\Omega(N)$.

Dimostrazione del Teorema 7.1

Perché il costo asintotico è necessariamente lineare?

- Essendo una query auto-contraddittoria, il punteggio massimo ottenibile per qualsiasi oggetto è esattamente 0.5.
- Se un algoritmo analizzasse meno di $N/2$ oggetti, lascerebbe almeno un oggetto x_0 **non valutato** ("intoccato").
- Un database teorico potrebbe quindi nascondere proprio in quell'oggetto non visto i gradi $\mu_Q(x_0) = 0.5$ e $\mu_{\neg Q}(x_0) = 0.5$, rendendolo la risposta corretta e ottimale.
- Poiché l'algoritmo non ha ispezionato l'intero database, mancherebbe questo oggetto e fornirebbe una risposta sbagliata.
- **Conclusione:** L'unico modo per garantire la correttezza matematica è leggere un numero lineare di tuple.